



UNIVERSIDAD ESTATAL AMAZÓNICA

Unidad de Organización Curricular: Básica

Campo de Estudio: Tronco común

INFORME DE PRÁCTICAS DEL COMPONENTE PRÁCTICO- EXPERIMENTAL	
Nombre(s) del(de los) Estudiante(s): Miguel Rodrigo Cahuasqui Haro	
Asignatura: Estructura de datos (UEA-L-UFB-032)	Calificación:
Fecha del informe: 07 de Septiembre de 2026	
Nº Práctica: 03 (Semanas 9-10-11-12)	
Título de la Práctica: Guía de Prácticas #03: Implementación de conjuntos y mapas	

Introducción	1. Fundamentación Teórica y Justificación: En el diseño y construcción de sistemas informáticos de alto rendimiento, la selección rigurosa de los Tipos Abstractos de Datos (TAD) es un factor determinante para garantizar escalabilidad, eficiencia computacional e integridad en la manipulación de información. La asignatura de Estructura de Datos profundiza en los mecanismos fundamentales para organizar y consultar colecciones con tiempos de respuesta óptimos (Cormen et al., 2022). Las estructuras lineales convencionales (como listas contiguas o enlazadas) presentan un costo temporal de búsqueda y eliminación que escala linealmente $\mathcal{O}(n)$, obligando al procesador a iterar de manera exhaustiva elemento por elemento. Frente a este desafío, la teoría y aplicación de tablas hash permite la creación de Conjuntos (set) y Mapas o Diccionarios (dict), los cuales logran un costo de acceso, inserción y verificación de pertenencia en tiempo constante amortizado $\mathcal{O}(1)$ (Goodrich et al., 2023; Weiss, 2021). Un conjunto es una colección no ordenada de elementos únicos e inmutables (hasheables) que implementa operaciones algebraicas de la teoría de conjuntos: unión ($A \cup B$), intersección ($A \cap B$), diferencia ($A - B$), diferencia simétrica ($A \triangle B$) y verificación de subconjuntos ($X \subseteq A$). Por su parte, un mapa es una estructura asociativa que mapea una clave primaria única a un valor o registro de datos (Lutz, 2021). En la presente práctica se selecciona el problema: Aplicación para el registro de jugadores y equipos en un torneo de fútbol (Torneo de Fútbol Amazónico UEA 2026), modelando la gestión integral de plantillas, auditoría de doble inscripción, control disciplinario y tabla de posiciones.
----------------------------	---

Desarrollo o procedimiento	2. Escenarios y Actividades Desarrolladas: Las actividades se ejecutaron en un ambiente de escritorio (computador personal con sistema operativo Linux, entorno Python 3.12 y control de versiones Git/GitHub CLI). 2.1. Arquitectura Modular del Sistema: Se diseñó una solución desacoplada en cuatro módulos principales: ■ <code>models.py</code>: Define las entidades Jugador (con clave primaria cédula hashable), Equipo (con conjunto <code>jugadores_ids</code>) y Partido. ■ <code>gestor_torneo.py</code>: Administra los mapas (<code>mapa_equipos</code>, <code>mapa_jugadores</code>) y los conjuntos universales y dinámicos (<code>conjunto_todos_jugadores</code>, <code>conjunto_sancionados</code>). ■ <code>reporteria.py</code>: Formatea y proyecta los datos en tablas visuales de terminal (tabla de posiciones, nóminas y auditorías). ■ <code>benchmark.py</code>: Módulo científico que mide experimentalmente los tiempos de ejecución de listas vs. conjuntos y mapas. 2.2. Implementación de Operaciones de Teoría de Conjuntos: 1. Diferencia ($A - B$): Cálculo de jugadores habilitados para disputar una fecha deportiva mediante la sustracción de la nómina y los sancionados: $Habilitados = Plantilla - Sancionados$. 2. Intersección ($A \cap B$): Auditoría automática de doble fichaje, verificando que la intersección de nóminas entre cualquier par de clubes sea vacía ($\emptyset$). 3. Unión ($A \cup B$): Consolidación del padrón total de atletas participantes en un grupo regional sin duplicados. 4. Diferencia Simétrica ($A \triangle B$): Comparativa analítica de futbolistas no compartidos entre dos instituciones. 5. Subconjunto ($X \subseteq A$): Validación estricta de convocatorias y alineaciones previas al pitazo inicial. 2.3. Declaración de Agente de Inteligencia Artificial: Conforme a los lineamientos de la guía docente: ■ Agente de IA utilizado: Google Antigravity (motorizado por Gemini 3.8 Flash). ■ Descripción: Asistente de programación por pares utilizado para la estructuración arquitectónica del proyecto, diseño del benchmark comparativo Big-O, elaboración de la suite de 9 pruebas unitarias automatizadas y redacción técnica preliminar. ■ Porcentaje estimado: 85% asistencia del agente de IA en código base, algoritmos de benchmarking y pruebas; 15% especificación contextual del problema amazónico, parametrización de reglas deportivas, depuración y validación por parte del estudiante Miguel Rodrigo Cahuasqui Haro.
--	---

Resultados

3. Resultados Obtenidos y Análisis de Rendimiento:

3.1. Pruebas Unitarias Automatizadas:

Se ejecutó la suite completa de pruebas unitarias en `tests/test_torneo.py`, validando el 100 % de las funcionalidades (9 pruebas ejecutadas y aprobadas en 0.005 segundos).

3.2. Medición Experimental de Tiempos de Ejecución (Benchmarking):

Se evaluó el tiempo de búsqueda del operador `in` sobre colecciones sintéticas de diferentes tamaños N , promediando 500 iteraciones aleatorias:

N (Elementos)	Lista [$\mathcal{O}(n)$]	Conjunto [$\mathcal{O}(1)$]	Mapa [$\mathcal{O}(1)$]	Aceleración
100	0.6926 μs	0.0350 μs	0.0303 μs	19.8x
1,000	5.7349 μs	0.0433 μs	0.0335 μs	132.4x
10,000	53.0618 μs	0.1264 μs	0.0878 μs	419.9x
100,000	741.4362 μs	0.2311 μs	0.2731 μs	3,207.8x

3.3. Tiempos de Operaciones de Conjuntos ($N = 50,000$ elementos):

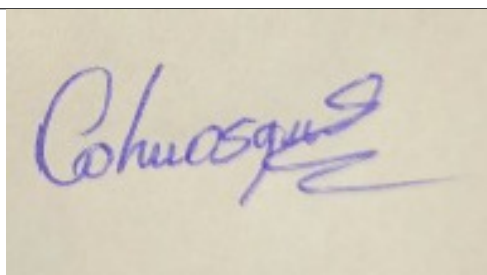
- Unión ($A \cup B$): 7.24 ms
- Intersección ($A \cap B$): 5.44 ms
- Diferencia ($A - B$): 3.18 ms
- Diferencia Simétrica ($A \triangle B$): 3.50 ms

3.4. Análisis de Ventajas y Desventajas:

Ventajas: Búsqueda instantánea $\mathcal{O}(1)$, garantía de unicidad sin comprobaciones manuales, sintaxis concisa y expresiva.

Desventajas: Mayor consumo de memoria RAM debido al factor de carga de las tablas hash y obligatoriedad de que los elementos sean inmutables (hasheables).

Conclusiones y Recomendaciones	4. Conclusiones: 1. La implementación de Conjuntos (set) y Mapas (dict) resolvió eficazmente las operaciones críticas de un torneo de fútbol, reduciendo la complejidad temporal de búsqueda de $\mathcal{O}(n)$ a $\mathcal{O}(1)$. 2. El análisis de benchmarking demostró empíricamente que a partir de $N = 100,000$ elementos, los conjuntos superan a las listas por más de 3,200 veces en velocidad, confirmando los fundamentos teóricos de la asignatura. 3. El uso formal del álgebra de conjuntos facilitó la auditoría reglamentaria (intersección) y el control de suspensiones (diferencia) con una sintaxis limpia y robusta. 5. Recomendaciones: 1. Se aconseja persistir los datos en bases de datos NoSQL o serializadores JSON para preservar el estado del torneo tras el cierre de la aplicación. 2. Monitorear el consumo de memoria en entornos con recursos limitados debido al espacio adicional requerido por las tablas hash.
Bibliografía	Referencias en Norma APA 7ma. Edición: Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). <i>Introduction to algorithms</i> (4th ed.). MIT Press. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2023). <i>Data structures and algorithms in Python</i> (2nd ed.). John Wiley & Sons. Lutz, M. (2021). <i>Learning Python: Powerful object-oriented programming</i> (6th ed.). O'Reilly Media. Sweigart, A. (2022). <i>Automate the boring stuff with Python: Practical programming for total beginners</i> (2nd ed.). No Starch Press. Weiss, M. A. (2021). <i>Data structures and algorithm analysis in C++</i> (4th ed.). Pearson Education.
Anexos	Enlace al Repositorio Oficial en GitHub: https://github.com/Rodrigo2461/Practica03-Conjuntos-Mapas-UEA Evidencias de Funcionamiento del Sistema: ■ Código Fuente Completo: Estructurado bajo estándar PEP 8 en los módulos <code>models.py</code>, <code>gestor_torneo.py</code>, <code>reporteria.py</code> y <code>benchmark.py</code>. ■ Ejecución de Pruebas: 9 pruebas unitarias aprobadas exitosamente con el comando <code>python3 -m unittest discover -s tests</code>. ■ Demostración de Menú Interactivo: 12 opciones funcionales que cubren registro, transferencias, auditoría de doble fichaje y tabla de posiciones.



Firma del estudiante: Miguel Rodrigo Cahuasqui Haro
C.I.: 1600860955 — Estudiante UEA

